Problem Statement 1

Cross-Encoder Based Semantic Retrieval and FAQ Matching System

Conversational AI (AIMLCZG521) — EC-1 Assessment

Total Marks: 10

General Instructions

1. Students must carefully read and follow the instructions provided in each problem statement and task.
2. Students must submit the completed assignment notebook in both formats:
    - Jupyter Notebook format (.ipynb)
    - PDF format of the executed notebook
3. The Jupyter Notebook must contain the output displayed for every executed cell.
4. Assignments developed or submitted using any Python IDE other than the prescribed environment will not be considered for grading.
5. Students must ensure that they submit the correct Assignment Set assigned to them. Submission of an incorrect Assignment Set will not be evaluated.
6. For each and every task, students must provide a detailed explanation, justification, and inference. Merely providing code and output will not be sufficient for evaluation.
7. The notebook should clearly include:
    - Assignment title
    - Student details (including contribution area for group assignments)
    - Problem statement
    - Dataset details and source
    - Tools and libraries used
    - Code implementation
    - Output screenshots/results
    - Explanation of the logic used
    - Justification for the chosen method/model/approach
    - Inference drawn from the results
    - Limitations observed
    - Possible improvements
    - Final conclusion
    - References

8. Students must clearly explain the architectural design, workflow, and reasoning behind the selected approach. This includes tokenization strategy, embedding generation, retrieval strategy, ranking/re-ranking, and evaluation.
9. Students must not simply use built-in APIs or pre-trained pipelines without explaining the underlying process. The implementation should clearly demonstrate conceptual understanding of the selected method.
10. Incomplete submissions, missing outputs, unclear explanations, absence of either the .ipynb file or the final PDF report, missing Virtual Lab screenshots, or lack of proper justification and inference may lead to a reduction of marks.
11. Students must prepare a final report in PDF format and upload it along with the .ipynb notebook. If the final PDF report is not attached, the submission will be considered partial, and marks will be reduced accordingly.
12. Students are advised to verify the complete notebook execution from start to end before submission. Notebooks with execution errors, missing cells, broken links, or unavailable datasets may be penalized.
13. Any copied content, direct use of AI-generated answers without proper understanding, or submission without meaningful explanation and inference may lead to mark deduction.
14. Proper references must be provided for datasets, models, libraries, research papers, APIs, documentation, or external resources used in the assignment.
15. Students must ensure that all figures, tables, architecture diagrams, workflow diagrams, and evaluation results are properly labelled and explained.
16. The final conclusion must summarize the key observations, strengths of the implemented approach, limitations, and possible future improvements.

Group Assignment Contribution Guidelines

This is a **group assignment**. Each team must clearly document the contribution of each member.

Contribution Area	Description
BPE Tokenizer Implementation	BPE tokenizer development, merge analysis, and optimization
Transformer Encoder Implementation	Multi-head attention, feed-forward network, residual connections, layer norm
Embedding Generation	Encoder model selection, pooling strategies, similarity analysis
BM25 & Sparse Retrieval	BM25 implementation, parameter tuning, TF-IDF comparison
Dense Retrieval & ANN	Embedding generation, ANN index building, parameter optimization
Hybrid Retrieval & RRF	RRF implementation, k-value experiments, fusion analysis
Cross-Encoder Re-ranking	Pre-trained Cross-Encoder usage, threshold optimization
Evaluation & Analysis	Metrics computation, analysis, and visualization
Visualization & Reporting	All visualizations, architecture diagrams, final report

Each member must contribute to **at least two areas**. The contribution of each member must be clearly stated in the notebook and final report.

Problem Statement

Build and evaluate a semantic retrieval and FAQ matching system that demonstrates the complete retrieval pipeline:

User Query → BM25 Keyword Retrieval → Dense Vector Retrieval → Hybrid Ranking (RRF) → Cross-Encoder Re-ranking → Match Decision

The system should leverage the concepts covered in the course:

- **BPE Tokenization** - from scratch
- **Transformer Encoder** - from scratch (Session 2, pages 6-8, 21)
- **Contextual Embeddings** - from encoder models (Session 2 + DistilBERT notebook)
- **Vector Similarity** - cosine, dot product, L2 distance (Session 2 + Similarity notebook)
- **ANN Search** - HNSW, IVF, PQ concepts (Session 2-3)
- **BM25 Sparse Retrieval** - with TF-IDF foundation (Session 3)
- **Hybrid Search with RRF** - combining dense and sparse (Session 3 + Similarity notebook)
- **Cross-Encoder Re-ranking** - final scoring using pre-trained model (Similarity notebook)

Module 1: Tokenization & Transformer Foundations

Task 1: Byte Pair Encoding Tokenizer with Optimization Analysis — 1.5 Marks

Use the Quora question-pair dataset: [https://huggingface.co/datasets/sentence-transformers/quora-duplicates]

Requirements:

1. Implement a custom **BPE tokenizer from scratch** (do not use pre-trained tokenizers).
2. Train on the Quora question-pair dataset with **three different vocabulary sizes**: 2,000, 5,000, and 10,000.
3. Include special tokens: [PAD], [UNK], [CLS], [SEP].
4. For each vocabulary configuration, report and analyze:
    - Vocabulary size and number of merge operations
    - Average tokens per sentence and compression ratio
    - Number of out-of-vocabulary (OOV) tokens on a held-out test set
    - Tokenization speed (tokens/second)
    - Effect on sequence length and computational cost
5. **Optimization Analysis:**
    - Analyze the merge sequence and identify the top 10 most frequent merge rules
    - Explain why certain character pairs were merged early
    - Discuss the trade-off: larger vocabulary → shorter sequences but more parameters vs smaller vocabulary → longer sequences but fewer parameters
6. Demonstrate tokenization on at least 10 sample questions, including:
    - Common words

- Rare terms
- Misspelled words
- Compound words

7. Include visualizations:
   - Token length distribution for each vocabulary size
   - Vocabulary growth curve (vocabulary size vs number of merges)
   - Merge frequency distribution

Inference:

- Which vocabulary size provides the best balance between compression and OOV handling?
- How does the choice of vocabulary size affect downstream retrieval performance?
- Justify your choice for the optimal vocabulary size for this task.

Task 2: Transformer Encoder Implementation — 1.5 Marks

Requirements:

1. Build a **Transformer Encoder from scratch** without using nn.Transformer, nn.TransformerEncoder, or nn.TransformerDecoder.
2. Implement the following components:
   - Multi-Head Self-Attention: With scaled dot-product attention
   - Feed-Forward Network: Two linear layers with ReLU activation
   - Residual Connections: Add input to output of each sub-layer
   - Layer Normalization: Apply after each sub-layer
   - Dropout: For regularization
3. The encoder should take:
   - Input token IDs
   - Segment IDs (to distinguish Question 1 vs Question 2)
   - Attention mask (for padding)
4. Use the [CLS] token representation for binary classification (duplicate or not).
5. Compare **two model configurations**:
   - Small: 2 layers, 4 heads, 128 embedding dimension
   - Medium: 4 layers, 8 heads, 256 embedding dimension
6. For each configuration, report:
   - Total parameter count
   - Training time per epoch

- Validation F1-score
- Inference latency per pair
- Memory usage

7. Demonstrate the forward pass on a sample pair, showing:
   - Input shape through each layer
   - Attention weight shapes
8. Visualize attention weights for at least one head (heatmap showing which tokens attend to which).

Inference:

- How does increasing model size affect performance and computational cost?
- What role does each component (self-attention, FFN, residual connections, layer norm) play?
- Why is the [CLS] token used for classification?

Module 2: Embeddings & Retrieval Models

Task 3: Contextual Embeddings and Similarity Analysis — 1.5 Marks

Requirements:

1. Load **three different** open-source encoder models:
   - DistilBERT (distilbert-base-uncased)
   - MiniLM (all-MiniLM-L6-v2)
   - BGE-small (BAAI/bge-small-en-v1.5) or GTE-small (thenlper/gte-small)
2. Generate embeddings for at least 20 question pairs from the Quora dataset using **all three models**.
3. **Polysemy Analysis:**
   - Select 5 ambiguous words (e.g., "bank", "cell", "apple", "spring", "light")
   - Find them in at least 3 different contexts
   - Compute pairwise cosine similarities between embeddings of the same word in different contexts
   - Explain how the embedding changes based on context
   - Compare how the three different encoder models handle polysemy
4. **Pooling Strategy Comparison:**
   - Implement three pooling strategies:
   - CLS token pooling
   - Mean pooling
   - Max pooling
   - For each model and pooling strategy:

- Report embedding dimension
- Compute similarity scores on 10 known similar/different pairs
- Identify which pooling strategy works best for duplicate detection

5. **Similarity Metric Comparison:**
   - Compute similarities using:
     - Cosine similarity
     - Dot product (with normalized vectors)
     - L2 distance (and convert to similarity)
   - Derive the mathematical relationship between L2 distance and cosine similarity for normalized vectors: $L2^2 = 2(1 - \text{cosine})$
   - Explain which metric is most appropriate and why
6. **Model Comparison:**
   - Compare the three models on:
   - Embedding quality (separation of duplicate/non-duplicate pairs)
   - Inference speed (queries per second)
   - Memory usage
7. **Visualization:**
   - Create a similarity matrix for 10 sample questions for each model
   - Create a bar chart comparing model performance

Inference:

- Why are contextual embeddings better than static embeddings (word2vec) for question matching?
- Which pooling strategy and similarity metric combination gives the best separation?
- Which model would you recommend for production and why?

Task 4: Sparse Retrieval with BM25 — 1.5 Marks

Requirements:

1. Create a FAQ collection (minimum **50 question-answer pairs**) from the Quora dataset.
2. Implement **BM25 retrieval** (can use rank-bm25 library).
3. **Formula Documentation:**
   - Write the complete BM25 formula:

     $$\text{score}(Q,D) = \Sigma \text{IDF}(qi) \cdot [f(qi,D)\cdot(k1+1)] / [f(qi,D) + k1\cdot(1-b+b\cdot|D|/\text{avgdl})]$$

- Explain each component:
  - **TF saturation**: (k1+1) * f / (f + k1 * norm)
  - **IDF**: log((N - df + 0.5) / (df + 0.5) + 1)
  - **Length normalization**: norm = 1 - b + b * (doc_len / avgdl)
- Explain parameters: k1 (default 1.5) and b (default 0.75)
- Explain **TF saturation** (Session 3, page 45)

4. **Parameter Sensitivity Analysis:**
   a. Experiment with 3 values of k1 (1.0, 1.5, 2.0)
   b. Experiment with 3 values of b (0.5, 0.75, 1.0)
   c. Report effect on Recall@5, Recall@10
   d. Identify optimal parameters
5. **BM25 vs TF-IDF Comparison:**
   a. Implement simple TF-IDF retrieval
   b. Compare on at least 10 queries
   c. Explain why BM25 outperforms TF-IDF
6. **Edge Case Analysis:**
   a. Test queries with: rare/unique terms, common terms, long/short queries, stopwords
7. **Visualization:**
   a. Plot Recall@10 vs k1 values
   b. Plot Recall@10 vs b values
   c. Show TF saturation curve

Inference:

- Why does BM25 handle rare/exact terms better than dense retrieval?
- How sensitive is BM25 to parameter choices?
- When would BM25 fail and why?

Task 5: Dense Retrieval & ANN Index Optimization — 1.5 Marks

Requirements:

1. Generate dense embeddings using the best encoder and pooling strategy from Task 3.
2. Build **at least two different ANN indices**:
   - HNSW
   - IVF or IVF-PQ

3. Implement cosine similarity using **normalized dot product**.
4. **Performance Comparison:**
    - Compare exact search (brute-force) vs each ANN index:
    - Query time (average, P95 latency)
    - Recall\@5, Recall\@10
    - Index build time
    - Memory usage
    - Create a comparison table
5. **Parameter Optimization:**
    - HNSW: Experiment with M (8, 16, 32) and ef_search (10, 50, 100, 200)
    - IVF: Experiment with nlist (10, 50, 100) and nprobe (1, 5, 10, 20)
    - Report optimal parameters for >90% recall with minimal latency
6. **Scalability Analysis:**
    - Extend FAQ collection to at least 100 documents
    - Measure performance scaling
    - Project performance for 10,000 documents using complexity formulas:
    - HNSW: O(log n)
    - IVF: O(KD + ND/K)
    - PQ: O(n·m)
7. **Visualization:**
    - Plot Recall\@10 vs Query Time
    - Plot Recall\@10 vs Memory Usage
    - Identify Pareto-optimal configurations

Inference:

- What is the accuracy vs speed trade-off for each ANN algorithm?
- Which algorithm provides the best balance for this task?
- How do the complexity formulas explain observed performance?

Module 3: Hybrid Retrieval & Re-ranking

Task 6: Hybrid Retrieval with RRF Optimization — 1.5 Marks

Requirements:

1. Retrieve top-20 from both BM25 (best parameters from Task 4) and dense retrieval (best ANN from Task 5).

2. Implement **Reciprocal Rank Fusion (RRF)**:

1. Implement **Reciprocal Rank Fusion (RRF)**:

   RRF(d) = Σ 1 / (k + rank_r(d))

   - Experiment with k = 10, 30, 60, 100, 200
   - Explain effect of k (Session 3, page 54)
   - Justify why k=60 is standard

2. **Retrieval Configuration Comparison:**
   - Compare 5 configurations:
     - BM25-only
     - Dense-only
     - Hybrid with k=10
     - Hybrid with k=60
     - Hybrid with k=100
   - Report: Recall@5, Recall@10, MRR, average query time
3. **Score Distribution Analysis:**
   - Plot score distributions from BM25 and dense retrieval
   - Explain why scores cannot be directly compared (Session 3, page 53)
   - Demonstrate how RRF solves this
4. **Consensus Analysis:**
   - Identify documents in top-10 of both systems
   - Show RRF "consensus bonus" (Session 3, pages 55-57)
   - Find example where a document ranked #5 in both outranks a document ranked #1 in one
5. **Visualization:**
   - Plot Recall@10 vs k values
   - Show side-by-side ranking comparison

Inference:

- Why does hybrid retrieval achieve better results than either method alone?
- When would dense retrieval fail and BM25 succeed?
- What is the optimal k value?
- How does the "consensus" principle explain RRF's effectiveness?

Task 7: Cross-Encoder Re-ranking & Threshold Optimization — 1.5 Marks

Requirements:

1. Use a pre-trained Cross-Encoder (cross-encoder/ms-marco-MiniLM-L-12-v2 or similar).
2. Re-rank the top candidates from hybrid retrieval.
3. **Ranking Comparison:**
    - Compare **three ranking stages**:
        - Initial BM25 ranking
        - Hybrid RRF ranking
        - Cross-Encoder final ranking
    - Show side-by-side for at least 5 queries
    - Identify cases where Cross-Encoder significantly changes ranking
4. **Threshold Optimization:**
    - Evaluate thresholds: 0.50, 0.60, 0.70, 0.80, 0.85, 0.90, 0.95
    - For each: compute Precision, Recall, F1
    - Identify **optimal threshold** using F1 maximization
    - Explain Precision-Recall trade-off
5. **Cost-Benefit Analysis:**
    - Measure Cross-Encoder inference time per query
    - Compare pipeline time with and without re-ranking
    - Calculate improvement in Recall@10 vs additional latency
    - Decide if re-ranking is worth the cost
6. **Visualization:**
    - Plot Precision, Recall, F1 vs Threshold
    - Show ranking changes before and after Cross-Encoder

Inference:

- Why is Cross-Encoder re-ranking more accurate than bi-encoder retrieval?
- What is the latency vs accuracy trade-off?
- When is re-ranking most beneficial? When is it unnecessary?
- What threshold provides the best balance for production?

Task 8: Comprehensive Evaluation, Production Recommendation & Contribution — 1.5 Marks

Requirements:

1. Evaluate the entire pipeline on **at least 30 test queries**.

2. **Multi-Stage Evaluation:**
    - **BM25-only**: Recall@5, Recall@10, MRR
    - **Dense-only**: Recall@5, Recall@10, MRR
    - **Hybrid (RRF)**: Recall@5, Recall@10, MRR
    - **Cross-Encoder re-ranking**: Accuracy, Precision, Recall, F1
    - **Final decision**: Confusion matrix
3. **Query Type Analysis:**
    - Categorize queries: Short (<5 words) vs Long (>10 words), Factual vs Opinion, Simple vs Complex
    - Report performance differences across categories
    - Explain why certain query types perform better/worse
4. **Production System Recommendation:**
   Based on your analysis, design a production-grade system including:
    - **Architecture Diagram**: Complete pipeline with all components
    - **Component Specifications**:
        - Optimal vocabulary size and tokenizer
        - Best encoder model and pooling strategy
        - Best ANN algorithm and parameters
        - Optimal BM25 parameters (k1, b)
        - Best RRF k value
        - Optimal matching threshold
        - Whether to use Cross-Encoder re-ranking
    - **Performance Estimates**: Expected latency, memory usage, accuracy
    - **Scalability Plan**: How the system scales to 100K, 1M, 10M documents
    - **Monitoring & Alerts**: What to monitor in production, when to alert
    - **Fallback Strategy**: What happens when the system fails or returns low-confidence results
5. **Model Comparison Summary:**
    - Create a comprehensive table comparing:
        - Your custom Transformer Encoder (from Task 2)
        - Three pre-trained encoder models (from Task 3)

- Compare on:
    - Embedding quality
    - Retrieval accuracy (Recall@10)
    - Inference speed
    - Memory usage
    - Training required
- Explain trade-offs and recommend the best model for production

6. **Team Contribution:**
    - Clearly document each team member's contribution
    - Map each member to specific tasks and deliverables
7. **Visualization:**
    - Complete pipeline architecture diagram
    - Performance comparison charts
    - Confusion matrix
    - Model comparison dashboard
8. **Limitations and Future Work:**
    - Identify at least 5 limitations
    - Propose improvements with more resources

Measurement Metrics

Category	Metrics
Tokenizer	Vocabulary size, Average/Max token length, Merges, OOV rate, Tokenization speed
Transformer Encoder	Parameters, Training time, Inference latency, Memory usage, Attention visualization
Embeddings	Embedding dimension, Pooling strategy comparison, Similarity computation time
Retrieval	Recall@5, Recall@10, MRR, Query time
ANN Search	Exact vs ANN latency, Recall@K, Memory usage, Index build time

Category	Metrics
BM25	Parameter sensitivity, TF-IDF vs BM25 comparison
Hybrid	RRF score, k value comparison, Improvement over individual methods
Re-ranking	Initial rank, Cross-Encoder score, Final rank, Re-ranking latency
Threshold	Precision, Recall, F1 at candidate thresholds
Decision	FAQ Match / RAG Route
Production	Latency budget, Memory budget, Scalability, Monitoring

Deliverables

1. Python notebook (.ipynb) containing implementation and executed results
2. PDF export of the notebook
3. Custom BPE tokenizer implementation with optimization analysis
4. Custom Transformer Encoder implementation with two configurations
5. Embedding generation and similarity analysis with three models
6. BM25 parameter sensitivity analysis
7. ANN comparison with multiple algorithms and parameters
8. Hybrid retrieval comparison with different k values
9. Cross-Encoder re-ranking results and threshold analysis
10. Comprehensive evaluation
11. Production system recommendation with architecture diagram
12. Model comparison summary table (custom Transformer + 3 pre-trained models)
13. Team contribution document

Experimental Constraints

1. **BPE tokenizer must be implemented from scratch** (do not use pre-trained tokenizers)
2. **Transformer Encoder must be implemented from scratch** (do not use nn.Transformer)
3. Use at least 3 open-source encoder models (DistilBERT, MiniLM, BGE-small, GTE-small)
4. Standard libraries allowed for BM25 (rank-bm25), FAISS/HNSW
5. Cross-Encoder can use pre-trained checkpoints
6. All hyperparameters and decisions must be documented
7. Results should be reproducible from the submitted notebook
8. Random seeds must be fixed and reported
9. All experiments must use the same test set for fair comparison
10. Team contributions must be clearly documented

References (From Course Materials)

1. Byte Pair Encoding (Session 1, pages 35-38; byte_pair_encoding.ipynb)
2. Transformer Architecture & Self-Attention (Session 2, pages 6-8, 21)
3. Transformer Encoder Models & Contextual Embeddings (Session 2, pages 9-10, 15-21; Embedding-distilbert.ipynb)
4. Vector Similarity Measures (Session 2, pages 30-31; Similarity Hybrid implementation.ipynb)
5. ANN Algorithms: HNSW, IVF, PQ (Session 2, pages 33-45; Session 3, pages 5-14, 17-32)
6. Sparse Retrieval: TF-IDF and BM25 (Session 3, pages 40-49)
7. Hybrid Search and RRF (Session 3, pages 50-58; Similarity Hybrid implementation.ipynb)
8. Cross-Encoder Re-ranking (Similarity Hybrid implementation.ipynb)
9. Complexity Analysis: O(KD + ND/K) for IVF, O(log n) for HNSW (Session 3, page 11, 33)